

RevSecureCloud — Knowledge Transfer (Technical)

Version: 3.0.0 Date: 2026-03-07

Purpose

This technical KT is targeted at **backend engineers and DevOps**. It covers architecture, all modules, the database schema, environment configuration, migration workflow, AI subsystems, enterprise governance layers (v3.0), seed scripts, test suite, and deployment runbooks.

System Architecture



Architecture pattern: Clean Architecture (Domain → Application → Infrastructure → Interface) **No ORM leakage into domain** — all database calls are in infrastructure or module service layers.

Tech Stack

Concern	Technology
Runtime	Node.js >= 20
Language	TypeScript 5.6
HTTP framework	Fastify 4
ORM	Prisma 5.22
Database (local)	PostgreSQL 17 via Docker
Database (production)	Neon PostgreSQL (serverless)
AI inference	Groq API (default: groq/llama-3.1-8b-instant)
Authentication	@fastify/jwt (access + refresh tokens)
Validation	Zod (env + request schemas)
Queue (optional)	BullMQ + Redis
Logging	Pino
API docs (dev)	@fastify/swagger + @fastify/swagger-ui (disabled in production)

Project Structure

src/	
app.ts	– Fastify app setup (plugins, routes, CORS, swagger)
server.ts	– Server bootstrap and graceful shutdown
config/	
env.ts	– Zod env schema – all env vars validated at startup
constants.ts	– App-wide constants
domain/	– Pure domain types (no Prisma, no HTTP)
connectors/	– ConnectorEvent, NormalizedSubscription, etc.
intelligence/	– AIInsightGenerator, AnomalyDetector, ForecastEngine,
RiskScoringEngine	
application/	
connectors/	– ConnectorEngine.ts, ProviderRegistry.ts
intelligence/	– RunIntelligenceAnalysisUseCase.ts
infrastructure/	
ai/	– OpenAIAdapter.ts (Groq-compatible)
connectors/stripe/	– Stripe event normaliser
connectors/github/	– GitHub event normaliser
data-migrations/	– Idempotent data-migration scripts
demo/	– DemoDataService.ts
modules/	
auth/	– login, refresh, JWT guards
users/	– profile CRUD
companies/	– company settings

- revenue/
 - revenue records + leakage detection
- audit/
 - audit trail read + write service
- billing/
 - Stripe webhook, plan info
- integrations/
 - provider connections, custom providers, icon presets
- ai/
 - AI insight generation, model catalogue, usage quota
 - ai-safety-logger.ts – safety checks + AiSafetyLog persistence
- agent/
 - domain/
 - enums/ActionType.ts
 - policies/AIPolicyEngine.ts ← NEW v3.0 – 8-rule policy engine
 - application/services/
 - AgentPlannerService.ts
 - AgentApprovalService.ts
 - AgentExecutionService.ts
 - DoubleApprovalService.ts ← NEW v3.0 – double-approval workflow
- jobs/
 - queues/
 - BullMQ queue definitions
 - workers/
 - BullMQ worker handlers
 - job-observability.ts ← NEW v3.0 – observable processor + JobLog lifecycle
- shared/
 - errors/
 - AppError, typed error codes
 - middleware/
 - authMiddleware.ts
 - tenant-context.ts ← NEW v3.0 – AsyncLocalStorage<TenantStore>
 - plugins/
 - prisma.plugin.ts – PrismaClient + tenant + slow-query middleware
 - request-tracing.plugin.ts ← NEW v3.0 – UUID-v4 X-Request-Id per request
 - tenant-context.plugin.ts ← NEW v3.0 – authenticated context in AsyncLocalStorage
 - tenant-enforcement.plugin.ts ← NEW v3.0 – Prisma auto-inject companyId
 - idempotency.plugin.ts ← NEW v3.0 – at-most-once POST/PUT/PATCH semantics
 - tenant-rate-limit.ts ← NEW v3.0 – plan-based rate limits
 - slow-query-monitor.ts ← NEW v3.0 – logs queries >200ms
 - utils/
 - response.ts – successResponse / errorResponse (now includes requestId + timestamp)
 - pagination.ts – page-based + cursor-based pagination helpers
 - dlq-retry.ts ← NEW v3.0 – executeWithRetry, sendToDeadLetterQueue, replay
- types/
 - fastify.d.ts – FastifyRequest aug: user, companyId, requestId
 - index.ts – Shared TS types (ApiResponse extends with requestId, timestamp)
- prisma/
 - schema.prisma – All 27 Prisma models
 - seed.ts – Bootstrap seed (demo company, users, data)
 - migrations/
 - 17 migrations
- scripts/
 - generate-doc-pdfs.js – Generate PDFs from docs/ using md-to-pdf
 - run-data-migrations.ts – Execute pending data-migration scripts
 - seed-10k.ts – Seed ~8k POC records locally
 - seed-revenue.ts – Seed revenue history
 - debug-ai-run.ts – Debug / dry-run AI engine
- tests/
 - unit/

```
ai-policy-engine.test.ts  - 14 tests
ai-safety-logger.test.ts  - 12 tests
dlq-retry.test.ts         - 6 tests
pagination.test.ts        - 13 tests
tenant-context.test.ts    - 4 tests
docs/
  KT_FUNCTIONAL.md
  KT_TECHNICAL.md
  TECHNICAL_AND_FUNCTIONAL.md
  prompts/ai-intelligence-backend.md
```

Environment Variables

All variables are validated at startup via `src/config/env.ts` (Zod). Missing required vars crash the process with a clear error.

Required

Variable	Example	Description
DATABASE_URL	postgresql://postgres:postgres@localhost:5432/revsecurecloud_dev	Prisma DB connection
JWT_SECRET	32+ char string	Access token signing secret
JWT_REFRESH_SECRET	32+ char string	Refresh token signing secret
APP_URL	http://localhost:3000	Base URL (CORS, OpenAPI)

Optional / Feature Flags

Variable	Default	Description
PORT	3000	HTTP listen port
HOST	0.0.0.0	HTTP listen host
API_PREFIX	/api/v1	Route prefix
REDIS_ENABLED	false	Enable BullMQ + Redis
REDIS_URL	—	Redis connection string
ENABLE_DEMO_DATA	false	Allow demo seed (blocked in production)
DEMO_COMPANY_SLUG	demo	Slug of the isolated demo tenant
GROQ_API_KEY	—	Groq inference API key (required for AI runs)
GROQ_MODEL	groq/llama-3.1-8b-instant	Default LLM model for AI engine

ENABLE_AI_ENGINE	true	Enable AI analysis endpoint
OPENAI_API_KEY	—	OpenAI key (only for enrichWithLLM: true)
OPENAI_MODEL	gpt-4o-mini	OpenAI model for LLM enrichment
OPENAI_MAX_TOKENS	400	Max tokens for LLM enrichment
PROD_DATABASE_URL	—	Neon production DB (used by migration scripts only — never in server)

Database Schema

27 models across 17 migrations. All tenant-scoped models carry `companyId` with a DB-level index for isolation.

Core Models

Model	Table	Description
Company	Company	Tenant root — subscription, plan, settings
User	User	Company user — role, passwordHash, preferences
RefreshToken	RefreshToken	JWT refresh tokens
Revenue	Revenue	Revenue records (period, amount, MRR/ARR, source)
RevenueLeakage	RevenueLeakage	Detected leakages — category, riskScore, resolvedAt
Invoice	Invoice	Stripe invoices
SubscriptionItem	SubscriptionItem	Stripe subscription objects
AuditLog	AuditLog	Immutable action log — actorId, before, after JSON snapshots
DataMigrationHistory	DataMigrationHistory	Tracks executed data-migration scripts

Integration Models

Model	Table	Description
Integration	Integration	Legacy v1 provider connection
ProviderConnection	ProviderConnection	V2 provider connection (encrypted config)
ConnectorEventRecord	ConnectorEventRecord	Events ingested by ConnectorEngine
RevenueEventLog	RevenueEventLog	Append-only revenue event stream
CustomProvider	CustomProvider	User-defined HTTP API integrations
ProviderIcon	provider_icons	Built-in icon presets for the icon picker
TestCredential	test_credentials	Dev/staging pre-fill credentials
ApiKey	ApiKey	Tenant API keys

AI Models

Model	Table	Description
AiModel	ai_models	Supported LLM catalogue (seeded via migration)
AiUsage	ai_usage	Per-company quota tracker (totalRuns, quotaRemaining)
AIInsight	ai_insights	Persisted analysis results (anomaly+forecast+risk)
AIModelUsage	ai_model_usage	Per-call LLM cost tracking (tokens, costUsd)
AIRecommendation	ai_recommendations	AI-generated recommendations (recommendationType, confidence, llmModel, promptVersion)

Agent Models

Model	Table	Description
AgentActionProposal	agent_action_proposals	AI-proposed actions — status, priorityScore, confidenceScore, estimatedImpact, payload
AgentExecutionLog	agent_execution_logs	Immutable execution history per proposal
ActionExecution	action_executions	Idempotent execution records — unique idempotencyKey

Operational Models

Model	Table	Description
FeatureFlag	feature_flags	Per-tenant feature toggles (companyId+feature unique)
ConnectorSyncLog	connector_sync_logs	Connector sync observability — recordsSynced, durationMs
JobLog	JobLog	Background job execution log (observableProcessor)

Enterprise Governance Models (NEW v3.0)

Model	Table	Description
ApiIdempotencyKey	api_idempotency_keys	DB-backed mutation dedup via x-Idempotency-Key header — 24-hour TTL
DeadLetterEvent	dead_letter_events	Persists connector events that exhausted retries for manual review + replay
ActionApproval	action_approvals	Double-approval workflow — requires two distinct approvers (approver1 ≠ approver2)
AiSafetyLog	ai_safety_logs	Records every AI prompt/completion with hallucination/injection/policy flags

Migration History (17 total)

#	Migration	Description
1	20260228120000	Add DataMigrationHistory
2	20260228162239	Add DataMigrationHistory (v2 naming)
3	20260228172119	Add AuditLog actor fields
4	20260228175936	Add User preferences (JSON column)
5	20260228204144	Add ConnectorEngine v2 models (ProviderConnection, ConnectorEventRecord, RevenueEventLog)
6	20260228234051	Add CustomProviders
7	20260228234459	Add ProviderIcons table
8	20260228234509	Seed built-in provider icons
9	20260301001011	Add TestCredentials table
10	20260301001027	Seed test credentials (Stripe, GitHub)
11	20260301002305	Seed remaining test credentials
12	20260301023926	Add AI engine models (AiModel, AllInsight, AIModelUsage)
13	20260301032302	Add AiUsage table
14	20260303070429	Add agent module (AgentActionProposal, AgentExecutionLog)
15	20260304192521	Add Copilot models (AIRecommendation, ActionExecution, FeatureFlag, ConnectorSyncLog, AuditLog before/after)
16	20260304200133	Add Enterprise Governance models (ApildempotencyKey, DeadLetterEvent, ActionApproval, AiSafetyLog)
17	20260304200134	Add JobLog table

Enterprise Governance Layer (NEW v3.0)

1. Request Tracing (`request-tracing.plugin.ts`)

Every HTTP request receives a UUID-v4 **request ID** before any handler runs.

- Stored on `request.requestId`
- Echoed back in `X-Request-Id` response header
- Injected into the Pino child logger — all log lines for a request share the same `requestId`
- Included in every error and success response envelope (`requestId` field)

2. Tenant Rate Limiting (`tenant-rate-limit.ts`)

Plan-based rate limits enforced per `companyId` :

Plan	Limit (req/min)
------	-----------------

STARTER	100
GROWTH	300
ENTERPRISE	1000
Default	100

- Uses `@fastify/rate-limit` with a custom `keyGenerator` (returns `companyId` from JWT context)
- Error code: `RATE_LIMITED` , HTTP 429, includes `retryAfter` seconds in response

3. Tenant Context Isolation (`tenant-context.plugin.ts` + `tenant-context.ts`)

Post-auth, the authenticated user's `{ companyId, userId, role }` is stored in `AsyncLocalStorage<TenantStore>` . Any downstream service can call `getTenantContext()` without threading params manually.

`SUPER_ADMIN` users bypass tenant scoping (returns `null` tenant context).

4. Tenant Enforcement Middleware (`tenant-enforcement.plugin.ts`)

A Prisma `$use` middleware auto-injects `companyId` into every `findMany` / `findFirst` / `findUnique` / `create` / `update` / `delete` on tenant-scoped models.

- Tenant-scoped models (explicit allow-list): `Revenue` , `RevenueLeakage` , `Invoice` , `SubscriptionItem` , `AuditLog` , `ProviderConnection` , `ConnectorEventRecord` , `RevenueEventLog` , `CustomProvider` , `ApiKey` , `AIInsight` , `AIModelUsage` , `AIUsage` , `AIRecommendation` , `AgentActionProposal` , `AgentExecutionLog` , `ActionExecution` , `FeatureFlag` , `ConnectorSyncLog` , `JobLog` , `ApiIdempotencyKey` , `DeadLetterEvent` , `ActionApproval` , `AISafetyLog`
- `SUPER_ADMIN` bypasses the middleware (no auto-inject)

5. Idempotency Guard (`idempotency.plugin.ts`)

For `POST` , `PUT` , and `PATCH` requests that include `X-Idempotency-Key` header:

1. **preHandler:** Look up `ApiIdempotencyKey` row. If found and status= `COMPLETED` , replay the cached response immediately (no handler runs).
2. Create pending row in `ApiIdempotencyKey` .
3. Handler runs normally.
4. **onSend:** Persist response body + status code in `ApiIdempotencyKey` row, mark status= `COMPLETED` .

TTL: 24 hours. Expired keys are not replayed.

6. Slow Query Monitor (`slow-query-monitor.ts`)

Prisma `$use` middleware that times every query. Queries exceeding **200 ms** are logged at `warn` level with:

```
{ "model": "Revenue", "action": "findMany", "durationMs": 347 }
```

7. AI Policy Engine (`AIPolicyEngine.ts`)

Evaluates AI agent actions against 8 built-in rules before execution. Returns a verdict:

Verdict	Meaning
---------	---------

ALLOW	Action passes all rules — proceed
REQUIRE_APPROVAL	Action requires double-approval before execution
DENY	Action is blocked — hard stop

Verdict precedence: DENY > REQUIRE_APPROVAL > ALLOW (most restrictive wins).

Built-in Rules (8):

Rule	Trigger	Verdict
no-bulk-delete	Bulk delete on 50+ records	DENY
no-demo-tenant-mutations	Any mutation in demo company	DENY
high-value-payment-retry	Payment retry > \$10,000	REQUIRE_APPROVAL
subscription-disable	DISABLE_SUBSCRIPTION action type	REQUIRE_APPROVAL
finance-notification	NOTIFY_FINANCE_TEAM with balance > \$50,000	REQUIRE_APPROVAL
low-confidence-block	Confidence score < 0.4	DENY
rate-limit-ai-runs	More than 10 AI runs/hour for company on STARTER plan	REQUIRE_APPROVAL
data-export-guard	Data export action type	REQUIRE_APPROVAL

Custom rules can be added by extending `AIPolicyEngine.addRule()` .

`buildContext()` fetches `{ plan, featureFlags }` from DB to provide rule inputs.

8. Double Approval Service (`DoubleApprovalService.ts`)

Implements a two-step human approval gate for `REQUIRE_APPROVAL` policy verdicts:



Methods:

- `initiate(actionId, requestedBy)` — creates `ActionApproval` row
- `submitApproval(approvalId, approverId)` — advances state; blocks if `approverId === approver1Id`

- `reject(approvalId, rejectedBy)` — sets status to `REJECTED`
- `isFullyApproved(approvalId)` — boolean check before execution
- `getApprovalStatus(approvalId)` — returns current `ActionApproval` record

9. AI Safety Logger (`ai-safety-logger.ts`)

Every LLM prompt/completion passes through `runSafetyChecks()` (pure function, no DB) which returns flags:

Flag	Check
<code>isHallucination</code>	Response contains uncertainty markers ("I think", "maybe")
<code>isPromptInjection</code>	Prompt contains injection patterns ("ignore previous", "jailbreak")
<code>isPolicyViolation</code>	Response content violates business policy rules

`logAISafety(prisma, params)` persists a row to `AiSafetyLog` with all flags + prompt/response text.

Verdict:

- Any `DENY` -level flag → `BLOCKED`
- Any warning flag → `WARNING`
- All clear → `SAFE`

10. DLQ & Retry (`dlq-retry.ts`)

Connector events that fail processing are retried with exponential backoff + jitter, then moved to the dead-letter queue:

```
executeWithRetry(fn, { maxRetries: 3, baseDelayMs: 1000 })
  → attempts: 1s, 2s, 4s (with ±20% jitter)
  → on exhaustion: sendToDeadLetterQueue() → DeadLetterEvent row
```

Functions:

- `executeWithRetry(fn, options)` — wraps any async function with retry semantics
- `sendToDeadLetterQueue(prisma, event)` — persists failed event to `DeadLetterEvent`
- `replayDeadLetterEvent(prisma, id)` — marks event for retry replay
- `resolveDeadLetterEvent(prisma, id)` — marks event as manually resolved

11. Job Observability (`job-observability.ts`)

BullMQ worker processors are wrapped with `observableProcessor()` which:

1. Creates a `JobLog` row with status `RUNNING` when job starts
2. Updates to `COMPLETED` on success (with `completedAt` and `result`)
3. Updates to `FAILED` on error (with `failedAt` and `errorMessage`)

`getJobStats(prisma, companyId)` returns aggregated job counts by status.

Module Deep-Dives

`auth` Module

- `auth.routes.ts` — `POST /login` , `POST /refresh`

- `auth.service.ts` — bcrypt verify, JWT sign (access 15m, refresh 7d), emits `LOGIN` audit event
- Guards: `authMiddleware` (JWT verify), `adminAuth` (`COMPANY_ADMIN+`), `roleGuard(roles[])` (any role set)

revenue Module

- `revenue.routes.ts` — CRUD for `Revenue` and `RevenueLeakage`
- `revenue.service.ts` — paginated queries with compound tenant filter
- Leakage resolution emits `RESOLVE_LEAKAGE` audit event with `before / after` snapshots

audit Module

- `audit.service.ts` — `createAuditLog(prisma, params)` write helper used across all modules
- `audit.routes.ts` — GET `/api/v1/audit` with `COMPANY_ADMIN` RBAC guard
- Supports `search` (icontains on `actorEmail/actorName/detail`) and exact `action` filter

integrations Module

- `integrations.routes.ts` — provider connections, custom providers, icon presets, test credentials
- `ConnectorEngine.ts` — normalises raw provider events → `ConnectorEvent` domain objects
- `ProviderRegistry.ts` — maps provider names to concrete adapter implementations

ai Module

- `ai.routes.ts` — `/run`, `/insights`, `/models`, `/usage`
- `RunIntelligenceAnalysisUseCase.ts` — orchestrates the 3 sub-engines + optional LLM enrichment
- **AnomalyDetector** — Z-score over rolling 30d window, flags spikes/drops
- **ForecastEngine** — linear trend projection with confidence bounds
- **RiskScoringEngine** — weighted composite of churn, dunning, leakage density → 0–100 score
- **OpenAIAdapter** — Groq-compatible OpenAI SDK wrapper; called only when `enrichWithLLM: true`
- Token usage written to `AIModelUsage`, company quota decremented in `AiUsage`
- **AI Safety Logger** — every LLM call result goes through `runSafetyChecks()` + `logAISafety()`

agent Module (Phase 4 + Enterprise)

Structure (Clean Architecture inside the module):

```
src/modules/agent/
  domain/
    enums/ActionType.ts           - 6 whitelisted action types
    policies/AIPolicyEngine.ts    - 8-rule policy engine (ALLOW/REQUIRE_APPROVAL/DENY)
  application/
    dto/agent.dto.ts             - Zod schemas + PlanResult, ProposalResponse types
    services/
      AgentPlannerService.ts      - Core proposal generation logic
      AgentApprovalService.ts     - Approve/reject state machine
      AgentExecutionService.ts    - Execute approved proposal (checks AIPolicyEngine)
      DoubleApprovalService.ts    - Two-step human approval gate
  infrastructure/
    ai/AgentAIClient.ts           - Groq prompt builder + response parser
    repositories/AgentProposalRepository.ts - Prisma wrapper
  interface/http/
    agent.controller.ts           - HTTP handler layer
    agent.routes.ts               - Fastify route registration
```

Proposal Generation Flow (AgentPlannerService.ts):

- 1. Resolve AIInsight (by insightId or latest for company)
- 2. Call AgentAIClient → LLM returns structured JSON proposals
- 3. Load existing PROPOSED proposals → build existingSignatures Set
- 4. **Payload-aware dedup:** signature = \${actionType}::\${payloadHash} — only exact duplicates skipped
- 5. Apply impactFloor (at least 100, derived from insight's leakage estimate × 10%)
- 6. Persist net-new proposals
- 7. Return { insightId, proposals, totalCreated, existingCount }

Execution Flow (AgentExecutionService.ts):

- 1. Load proposal — must be APPROVED
- 2. Call AIPolicyEngine.evaluate() with built context
- 3. If verdict = DENY → return 403 with policy rule details
- 4. If verdict = REQUIRE_APPROVAL → initiate DoubleApprovalService.initiate() , return 202
- 5. If verdict = ALLOW → execute action, persist AgentExecutionLog , emit AGENT_PROPOSAL_EXECUTED audit

AI Prompt Configuration (AgentAIClient.ts):

- Model: GROQ_MODEL env (default: groq/llama-3.1-8b-instant)
- Temperature: 0.2 (low randomness for consistent structured output)
- Max tokens: 2048
- Count hints (risk-driven): CRITICAL=5-7, HIGH=4-6, MEDIUM=3-5, LOW=2-3
- Multiple proposals of same actionType allowed if targeting different segments

Test Suite (Vitest)

49 tests across 5 files — all passing.

Framework: Vitest 4.x

Config: vitest.config.ts — coverage via v8, test pattern tests/**/*.test.ts

File	Tests	Coverage Focus
ai-policy-engine.test.ts	14	DENY/ALLOW/REQUIRE_APPROVAL verdicts, custom rules
ai-safety-logger.test.ts	12	Hallucination detection, injection, policy flags
dlq-retry.test.ts	6	Retry logic, exhaustion, onExhausted callback
pagination.test.ts	13	Page-based + cursor-based helpers
tenant-context.test.ts	4	AsyncLocalStorage isolation, SUPER_ADMIN bypass

Run tests:

```
npm test           # single run
npm run test:watch  # watch mode
npm run test:coverage # v8 coverage report
```

Local Development Quickstart

```
# 1. Install dependencies
npm ci

# 2. Start local Postgres
docker compose up -d

# 3. Copy env file
cp .env.example .env # fill in JWT_SECRET, JWT_REFRESH_SECRET

# 4. Apply migrations + generate client
npx prisma migrate dev
npx prisma generate

# 5. Seed demo data
npx tsx prisma/seed.ts

# 6. Start dev server (tsx watch)
npm run dev
# → http://localhost:3000
# → http://localhost:3000/docs (Swagger UI, dev only)
```

Production Deployment (Neon)

Apply migrations to production

```
$env:DATABASE_URL = "<PROD_DATABASE_URL>"
npx prisma migrate deploy
$env:DATABASE_URL = $null
```

Or use the PowerShell one-liner (reads from `.env`):

```
$prodUrl = ((Get-Content .env -Raw) -split "`n" | Where-Object { $_ -match
"^PROD_DATABASE_URL=" } | Select-Object -First 1).Trim() -replace "^PROD_DATABASE_URL=", "";
$env:DATABASE_URL = $prodUrl; npx prisma migrate deploy 2>&1; $env:DATABASE_URL = $null
```

Seed production data

```
# 10k records
$env:DATABASE_URL = $prodUrl; npx tsx scripts/seed-10k.ts; $env:DATABASE_URL = $null

# AI showcase data (with WIPE=true to clear first)
$env:DATABASE_URL = $prodUrl; $env:WIPE = "true"; npx tsx scripts/seed-ai-showcase.ts;
$env:DATABASE_URL = $null; $env:WIPE = $null
```

Verify migration status

```
npx prisma migrate status
```

Data Migration Framework

Separate from schema migrations — used for backfilling data without touching Prisma schema.

- **Runner:** `scripts/run-data-migrations.ts`
- **Script location:** `src/infrastructure/data-migrations/*.ts`
- **Each script exports:** `export async function run(prisma: PrismaClient): Promise<void>`
- **Idempotency:** Scripts are recorded in `DataMigrationHistory` table; already-run scripts are skipped
- **Run:** `npm run db:migrate:data`

Example script: `001_backfill_audit_logs.ts` — backfills `actorEmail` / `actorName` on legacy rows.

Seed Scripts Reference

Script	Command	Records	Target
<code>prisma/seed.ts</code>	<code>npx tsx prisma/seed.ts</code>	Demo company + users	Local
<code>scripts/seed-10k.ts</code>	<code>npm run seed:10k</code>	~8k across 6 tables	Local
<code>scripts/seed-10k.ts</code>	<code>npm run seed:10k:wipe</code>	Wipe then ~8k records	Local
<code>scripts/seed-revenue.ts</code>	<code>npx tsx scripts/seed-revenue.ts</code>	Revenue history	Local
<code>scripts/seed-ai-showcase.ts</code>	see above PowerShell	AI-ready showcase data	Production

AI Subsystem — Configuration Details

```
AI Provider: Groq (OpenAI-compatible SDK)
Default Model: groq/llama-3.1-8b-instant
Temperature: 0.2
Max Tokens: 2048
```

LLM flow:

1. `POST /api/v1/ai/run` hits `RunIntelligenceAnalysisUseCase`
2. Three sub-engines run in parallel (no LLM yet)
3. Result is saved as `AIInsight`
4. If `enrichWithLLM: true` → `OpenAIAdapter.summarise()` called → `AIInsight.summary` and `llmEnriched` updated
5. Result passes through `runSafetyChecks()` — flags stored in `AISafetyLog`
6. Token usage saved to `AIModelUsage` for cost tracking

Quota: `AiUsage.quotaRemaining` decremented on each run. Quota reset mechanism is a manual admin operation or billing event hook (out of scope currently).

Demo Data System

- Location: `src/infrastructure/demo/DemoDataService.ts`
- **Guards:** throws if `NODE_ENV === 'production'` or `ENABLE_DEMO_DATA !== 'true'`
- **Isolation:** company with `slug === DEMO_COMPANY_SLUG`
- **Idempotency:** `upsert` prevents duplicates on repeated runs

TypeScript & Build

```
# Type check (no emission)
npx tsc --noEmit

# Dev mode (tsx watch)
npm run dev

# Build (esbuild, if configured)
npm run build
```

The `npm run dev` command uses `tsx --watch` for zero-config TypeScript execution with hot reload.

Observability & Logging

- **Logger:** Pino (structured JSON in production, pretty-print in dev)
- **Request ID:** Every log line for a request includes `requestId` field
- **Health endpoint:** `GET /health` — checks Prisma DB connectivity
- **Slow query logging:** Queries >200ms logged at `warn` level with model, action, durationMs
- **AI cost tracking:** `AIModelUsage` table — query by `companyId` + date range for cost reports
- **AI safety auditing:** `AISafetyLog` table — every LLM call with safety verdicts
- **Agent execution tracking:** `AgentExecutionLog` — one row per execution attempt
- **Job tracking:** `JobLog` table — every background job with lifecycle status
- **DLQ:** `DeadLetterEvent` table — failed connector events awaiting replay

Security Architecture

Concern	Implementation
Authentication	JWT (access 15m, refresh 7d)
Authorisation	Role-based guards on every protected route
Tenant isolation	<code>companyId</code> Prisma middleware auto-inject on all tenant-scoped queries
Rate limiting	<code>@fastify/rate-limit</code> — plan-based per <code>companyId</code> (100/300/1000 req/min)
Security headers	Helmet (<code>@fastify/helmet</code>)
CORS	Explicit allowed origins; allowed headers include <code>X-Idempotency-Key</code> , <code>X-Request-Id</code>
Idempotency	<code>X-Idempotency-Key</code> header — DB-backed 24h dedup for POST/PUT/PATCH

Request tracing	X-Request-Id UUID-v4 on every response for correlation
AI safety	runSafetyChecks() on every LLM response; AiSafetyLog persists all flags
AI policy	AIPolicyEngine 8-rule evaluation before agent action execution
Double approval	DoubleApprovalService — two distinct human approvers required for high-risk actions
Stripe webhook	Raw body preserved via raw-body.helper.ts + signature verification
Demo seeding	Explicitly blocked in NODE_ENV=production
Production secrets	Must use environment secrets (not .env file) — PROD_DATABASE_URL, JWT_SECRET, GROQ_API_KEY

Critical quirk: The global raw-body plugin (Stripe webhook support) rejects POST requests with no body. Always send {} as minimum body on all POST endpoints.

Troubleshooting

Symptom	Likely Cause	Fix
npm run dev exits immediately	Port conflict or missing .env var	Check .env; run netstat -ano findstr :3000
POST /agent/plan returns 500	Body missing from request	Send {} as body
POST /ai/run returns 500	GROQ_API_KEY not set	Add to .env
Migration fails locally	Schema drift / stale volume	docker compose down -v; docker compose up -d; npx prisma migrate dev
GET /api/v1/audit returns 403	Caller is ANALYST or VIEWER	Use COMPANY_ADMIN token
existingCount > 0, totalCreated = 0	Dedup blocked all proposals	Approve/reject/execute existing proposals or use cleanup endpoint
POST returns 409 with DUPLICATE_REQUEST	Idempotency key already used	Use a new X-Idempotency-Key value for a new request
429 RATE_LIMITED response	Exceeded plan rate limit	Respect retryAfter in response; upgrade plan for higher limits
Agent execute returns 403 POLICY_DENY	AIPolicyEngine blocked the action	Review policy verdict details in response; adjust proposal
Agent execute returns 202 APPROVAL_REQ	AIPolicyEngine requires double-approval	Complete double-approval workflow via ActionApproval endpoints
Neon DB P1001 connection error	Cold start (serverless)	Retry after 2–3 seconds

Contacts & Ownership

- **Backend:** development team
- **Production DB:** Neon project `ep-young-glade-a1hfm4jg.ap-southeast-1.aws.neon.tech`
- **AI Provider:** Groq — API key in production env secrets
- **Deployment target:** Neon (DB), Render or similar (API server)